

GCP Real-Time Usage Analytics Pipeline

Complete Study Module — Project Deep Dive

Sanket Navnath Janger | Binghamton University, SUNY | MS Computer Science (AI Track)

github.com/SanketJanger/usage-analytics-pipeline | May 2026

Section	Topic
1	Tools Used + Alternatives
2	Problem Statement
3	Traditional Approach — How It Was Done Before
4	Why This Project — Motivation
5	What Makes This Different
6	Project Flow — Step by Step
7	Architecture Decisions — Why This, Not That
8	Outcomes and Results
9	Real-World Connections
10	How to Explain This in an Interview

Section 1 — Tools Used and Their Alternatives

Every tool in this pipeline was chosen deliberately. Understanding what each tool does, why it was chosen, and what the alternatives are is critical — both for interviews and for making good engineering decisions in the future.

Python 3.11	Event generation, Pub/Sub publishing, BigQuery writing, FastAPI backend	Go, Java, Node.js	Fastest to write, richest GCP SDK, best for data work. Go was considered for performance but Python was sufficient.
Cloud Pub/Sub	Async message transport between publisher and processor. Decouples producer from consumer.	Apache Kafka, AWS SQS, RabbitMQ	Native GCP, zero infrastructure to manage, auto-scales, integrates directly with Eventarc for Cloud Function triggers.
Cloud Functions Gen2	Serverless event processor. Triggered automatically on each Pub/Sub message.	Cloud Run (always-on), GKE, Compute Engine	No server management, scales to zero, pay-per-use. Gen2 uses Cloud Run under the hood — faster cold start than Gen1.
BigQuery	Columnar analytical warehouse. Stores all events for SQL analytics.	PostgreSQL, AWS Redshift, Snowflake	Native GCP, handles petabytes, serverless, streaming inserts API, built-in partitioning. Free tier generous.
FastAPI	REST API layer. Exposes BigQuery data as live HTTP endpoints.	Flask, Django, Express.js	Fastest Python framework, auto-generates Swagger docs, async support, type hints reduce bugs.
Cloud Run	Serverless container hosting for the FastAPI app.	App Engine, GKE, Compute Engine	Zero server management, scales to zero, deploy from source with one command, public URL out of the box.
Looker Studio	BI dashboard natively connected to BigQuery.	Tableau, Power BI, Grafana	Free, native BigQuery connector, no extra infrastructure, shareable public link in one click.
Eventarc	GCP event routing layer. Routes Pub/Sub messages to the Cloud Function trigger.	Manual push subscriptions	Fully managed, handles auth between Pub/Sub and Cloud Run automatically. Required for Gen2 functions.
Docker	Containerizes the FastAPI app for Cloud Run deployment.	Buildpacks	gcloud run deploy --source uses Docker automatically. Single Dockerfile controls the build.

GCP IAM	Identity and access management. Controls which service accounts can do what.	AWS IAM, manual auth	Least-privilege security. Each service only has exactly the permissions it needs — nothing more.
----------------	--	----------------------	--

Key Insight on Tool Selection

The most important tradeoff in this stack is managed vs self-managed. Every tool chosen is fully managed by Google — you don't run servers, you don't manage scaling, you don't patch operating systems. The tradeoff is less control but dramatically faster development and lower operational overhead. For a project at this scale, managed services are almost always the right call.

Section 2 — Problem Statement

Core Question

How do you know what's happening in your product right now — not yesterday, not an hour ago, but this second?

Every product with users has this problem. You have thousands (or millions) of people clicking buttons, making purchases, hitting API endpoints — and you have almost no visibility into what's actually happening unless you go look at logs manually.

Specific problems this pipeline solves:

- Which features are users actually using? Most products are built on assumptions about what users want. Real event data replaces assumptions with facts.
- Which API endpoints are failing right now? A spike in 5xx errors on `/api/checkout` could be losing the company thousands of dollars per minute. Without real-time monitoring, you find out when customer support gets flooded — too late.
- What does API latency look like at the 95th and 99th percentile? Average latency hides the worst-case user experience. p99 of 2000ms means 1 in 100 users is waiting 2 seconds for checkout to load.
- Which components need to scale up before traffic hits? Knowing that checkout traffic spikes every day at 6pm lets you pre-warm servers — instead of scrambling after the slowdown starts.
- How do you detect a bad deploy instantly? If error rate on an endpoint goes from 1% to 35% after a new release, you want to know in seconds — not after users start complaining on Twitter.

Why It Matters

These are not academic problems. Every company above a certain size has dedicated teams solving exactly these problems — Site Reliability Engineers (SRE), Data Engineers, Platform Engineers. The pipeline in this project is a smaller version of what those teams build.

Section 3 — Traditional Approach — How It Was Done Before

Before real-time event pipelines, companies did this:

3.1 Log Files + Manual Analysis

Every server writes log files — text files with timestamps recording what happened. To understand user behavior, engineers would download these log files, write scripts to parse them, and generate reports. This could take hours or days. By the time you had the report, the problem was already over — or worse, it had been going on for days without anyone noticing.

Example

A bug is deployed at 2pm. Users start getting errors. The log analysis job runs at midnight. Engineers see the report at 9am the next morning — 19 hours of errors, thousands of failed transactions.

3.2 Batch ETL Pipelines

Companies would run nightly batch jobs — Extract, Transform, Load — that pulled all the day's data, cleaned it up, and loaded it into a data warehouse. This gave you dashboards, but they were always showing yesterday's data. Decision-making was inherently backward-looking.

- Data warehouse updated once per night
- Reports always based on data that is at least 8-24 hours old
- Any real-time decision (pricing, scaling, alerting) was impossible
- Engineers had to manually check dashboards — no auto-alerting

3.3 Database Polling

Some systems would have applications write events directly to a relational database (PostgreSQL, MySQL), and then polling jobs would query the database every few minutes for new records. This worked at small scale but created serious problems as volume grew — the database became a bottleneck, queries slowed everything down, and the system would fall over under load.

3.4 The Core Problems with All Traditional Approaches

Latency	Hours to days between event and insight	Seconds to minutes
Scalability	Database falls over under load	Pub/Sub + BigQuery scale infinitely
Reliability	If the batch job fails, you lose the window	Dead letter queue — nothing lost
Alerting	Manual checks or cron-based polling	Automatic threshold-based alerts

Cost	Large always-on servers processing batch jobs	Pay only when processing — scale to zero
-------------	---	--

Section 4 — Why This Project — Motivation

Personal Motivation

The motivation was direct and honest. Rakuten Advertising's internship role specifically mentioned building usage tracking pipelines, standardizing data formats, and working with Pub/Sub and BigQuery. Rather than just listing GCP on a resume from coursework, the goal was to actually build and run the exact type of system the role requires — so that in an interview, the conversation could be about real engineering decisions made from experience, not hypotheticals.

Technical Curiosity

Most tutorials teach individual components in isolation — here's how Pub/Sub works, here's how BigQuery works. What they don't teach is how to connect everything together, debug the gaps (like IAM permission failures that drop events silently), and build something that actually runs end-to-end with real data. Building this project forced an understanding of the entire system, not just individual pieces.

Portfolio Gap

The Cloud Computing course covered GCP conceptually and included a data migration project, but there was nothing live and publicly accessible. A recruiter clicking the GitHub profile needed to see a running project — not just screenshots from a course. This pipeline is live right now. The dashboard, the API, the BigQuery data — all of it is real and accessible to anyone.

The 'Why' in One Sentence

I wanted to understand the infrastructure that companies like Rakuten use to turn raw user behavior into business decisions in real time — and the only way to really understand it was to build it myself.

Section 5 — What Makes This Different

Compared to Other Student Projects

- It's live and public. Most projects are screenshots or local demos. This one has a public URL that anyone can open right now and see real data flowing through it.
- It's end-to-end. Most tutorials build one layer — a publisher, or a BigQuery loader, or a dashboard. This connects all layers together with real inter-service communication.
- It has production-grade reliability features. Dead letter queue, IAM least-privilege, Cloud Monitoring alerts — these are things production systems have, not tutorials.
- It's decision-oriented. The analytics queries aren't just 'count events' — they compute p95/p99 latency, error rate breakdowns, user engagement per component. These are the metrics engineering teams actually use.
- It has a real API. The FastAPI layer means any application can consume the pipeline's data. It's not just a visualization — it's infrastructure.

Compared to Traditional Batch Pipelines

- Sub-2-second end-to-end latency vs hours for batch jobs.
- Events are never lost — dead letter queue catches failures vs batch jobs that silently drop bad records.
- Auto-alerting fires in real time vs waiting for a morning report.
- Serverless — scales to zero when idle, scales up instantly under load vs always-on batch servers.
- Cost-optimized queries — DATE partitioning and component clustering mean BigQuery only scans relevant data slices vs full table scans.

The Differentiator

The combination of: (1) real-time streaming, (2) dead letter queue for reliability, (3) partitioned warehouse, (4) REST API layer, and (5) live public dashboard — all in one project — is what makes this production-grade rather than a tutorial exercise.

Section 6 — Project Flow — Step by Step

Step 1 — Event Generation (publisher.py)

The Python publisher simulates a live product with 6 components: checkout, search, product-listing, recommendations, ad-banner, and user-profile. Every 500ms it generates one of two event types:

- UI Interaction events — captures what users are doing in the interface. Fields: event_id (UUID), event_type='ui_interaction', component, user_id, session_id, timestamp, metadata (JSON with action type like 'button_click', 'page_view', 'scroll').
- API Request events — captures backend performance. Same fields plus: latency_ms (response time), status_code (200/400/404/500), endpoint (/api/checkout, /api/search, etc.), HTTP method.

The status code distribution is weighted to simulate realistic traffic: 200 (70%), 201 (10%), 400 (8%), 404 (7%), 500 (5%). This intentionally creates error rates the monitoring system can detect and alert on.

Each event is serialized as JSON, encoded as bytes, and published to the Pub/Sub topic using the google-cloud-pubsub Python client. The publisher.publish() call returns a Future — the message is not confirmed sent until future.result() is called.

Key Concept

The publisher is completely decoupled from what happens next. It doesn't know or care whether the Cloud Function is running, whether BigQuery is healthy, or whether anyone is consuming the messages. It just sends to Pub/Sub and that's it. This decoupling is the core architectural principle.

Step 2 — Message Transport (Cloud Pub/Sub)

Google Cloud Pub/Sub is a fully managed async messaging service. It sits between the publisher and the Cloud Function and provides three critical guarantees:

- Durability — messages are stored on Google's infrastructure until they are successfully processed. If the Cloud Function is down, messages queue up and replay automatically when it comes back.
- At-least-once delivery — every message will be delivered at least once. The consumer must handle duplicates (which BigQuery's streaming insert API handles gracefully with event_id as a natural deduplication key).
- Dead letter forwarding — if the Cloud Function fails to process a message after 5 attempts (configured on the Eventarc subscription), the message is forwarded to the usage-events-dead-letter topic instead of being silently dropped or retried indefinitely.

The topic (usage-events) receives published messages. The Eventarc subscription listens on the topic and triggers the Cloud Function on each new message. This trigger mechanism is handled entirely by GCP — no polling, no cron jobs, no manual wiring.

Step 3 — Stream Processing (Cloud Function Gen2)

The Cloud Function (main.py) is a serverless Python function that wakes up automatically on each Pub/Sub message. Here's exactly what it does:

- Decodes the message — Pub/Sub messages arrive as base64-encoded bytes. The function decodes them to get the raw JSON string.
- Parses the JSON — converts the JSON string to a Python dictionary.
- Normalizes the timestamp — Python's `datetime.isoformat()` produces a string like `'2026-03-25T01:18:43.924+00:00'`. BigQuery expects `'YYYY-MM-DD HH:MM:SS.ffffff'`. The function strips the timezone suffix and trims microseconds to 6 digits.
- Validates required fields — checks that `event_id`, `event_type`, `component`, `user_id`, `session_id`, and `timestamp` are all present and non-null.
- Writes to BigQuery — calls `client.insert_rows_json()` with a single-row list. This is BigQuery's streaming insert API — rows are queryable within 1-2 seconds of insertion.
- Logs the result — INFO log on success, ERROR log on BigQuery insert failure. These logs are visible in Cloud Logging and were critical for debugging.

IAM Lesson Learned

The Cloud Function runs as the `pipeline-sa` service account. This account needs: `pubsub.subscriber` (receive messages), `bigquery.dataEditor` (write rows), `run.invoker` (allow Eventarc to invoke the function), `eventarc.eventReceiver` (accept Eventarc triggers). Missing any one of these caused silent failures — events received but never written. Debugging this required reading Cloud Logging, not error messages.

Step 4 — Storage (BigQuery)

All events land in a single BigQuery table: `usage-analytics-pipeline.usage_analytics.usage_events`. The schema design was the most important architectural decision — one table for both UI events and API events, with nullable fields for the fields that only apply to one type.

- DATE partitioning on timestamp — BigQuery physically separates data by day. A query for 'last 24 hours' scans only today's partition instead of the entire table. At 10 million events, this is the difference between a \$0.05 query and a \$5 query.
- Clustering by component, event_type — within each partition, rows are physically sorted by component then event_type. Queries filtering on these columns skip irrelevant row groups entirely.
- Streaming inserts — BigQuery's `insert_rows_json` API accepts rows immediately and makes them queryable within 1-2 seconds. This is different from batch loading which requires files in GCS and has higher latency.

Step 5 — API Layer (FastAPI on Cloud Run)

The FastAPI application (`api/main.py`) is deployed as a Docker container on Cloud Run. It exposes five endpoints, each running a different BigQuery query and returning JSON:

- GET `/health` — queries total event count and last event timestamp. Shows pipeline is alive. Response includes `checked_at` timestamp so you can verify freshness.
- GET `/events/volume?hours=24` — events grouped by component and event_type with percentage of total. hours parameter is configurable from 1 to 168 (one week). Shows which parts of the product are most active.
- GET `/events/errors?hours=24` — per-endpoint error rate breakdown (4xx vs 5xx). Includes `alert: true/false` flag and `flagged_endpoints` list when any endpoint exceeds 30% error rate. This is the auto-alerting logic.

- GET /events/latest?limit=20 — most recent N events ordered by timestamp DESC. Used to power the live event feed on the dashboard.
- GET /events/users?hours=24 — unique users and events per user per component. Measures engagement depth — not just traffic, but how deeply users interact with each feature.

The /dashboard route serves the static HTML dashboard file. Cloud Run handles HTTPS, load balancing, and auto-scaling automatically.

Step 6 — Live Dashboard (Chart.js)

The dashboard (api/static/dashboard.html) is a single HTML file served by FastAPI. It makes fetch() calls to the FastAPI endpoints every 10 seconds and updates the UI with the results. Key components:

- Pipeline flow animation — animated dots moving along the arrows between components (Python → Pub/Sub → Cloud Function → BigQuery → FastAPI) to show the pipeline is alive.
- Stat cards — total events ingested, pipeline health status, highest error rate endpoint, active component count.
- Bar chart — event volume by component using Chart.js. Updates every 10 seconds.
- Doughnut chart — UI interactions vs API requests split.
- Error rate table — each endpoint with a color-coded bar (green < 20%, orange 20-30%, red > 30%) and request count.
- Live event feed — scrolling list of the 15 most recent events showing event type, component, status code, latency, and timestamp.
- Alert banner — automatically appears when any endpoint crosses 30% error rate, listing the flagged endpoints.

Step 7 — Reliability: Dead Letter Queue

The dead letter queue is configured on the Eventarc subscription (the subscription that connects Pub/Sub to the Cloud Function). If the Cloud Function fails to process a message — throws an exception, crashes, times out — Pub/Sub retries delivery. After 5 failed attempts, instead of retrying indefinitely (which would block the queue) or silently dropping the message, Pub/Sub forwards it to the usage-events-dead-letter topic.

A separate subscription (usage-events-dead-letter-sub) stores these failed messages. In a production system you would have a separate consumer reading from this topic, logging the failures, and alerting on-call engineers. For this project, the dead letter topic acts as a permanent store of bad events that can be investigated and replayed once the underlying bug is fixed.

Step 8 — Monitoring and Alerting (Cloud Monitoring)

A Cloud Monitoring alert policy watches the Cloud Run service metrics. The condition monitors the rate of non-2xx responses (error rate) on the usage-analytics-api Cloud Run service. When this rate exceeds 30% sustained for 60 seconds, the policy fires a notification to the email channel (sjanger@binghamton.edu).

The alert policy was created via the gcloud CLI using a JSON policy definition file, making it version-controlled and reproducible. This is the same approach SRE teams use — infrastructure as code for monitoring configuration.

Section 7 — Architecture Decisions — Why This, Not That

Single table for both event types

Alternative Two separate tables (ui_events and api_events)

One table with nullable fields eliminates joins in all analytics queries. The tradeoff is some null columns for each event type, but this is negligible in a columnar store like BigQuery where nulls are essentially free.

Cloud Functions Gen2 over Cloud Run (dedicated)

Alternative A long-running Cloud Run service consuming from Pub/Sub

For event-driven processing, serverless functions are ideal — they scale to exactly the right number of instances automatically, scale to zero when idle, and have zero infrastructure management. A dedicated consumer would need manual scaling logic.

Pub/Sub over direct API calls

Alternative Publisher calls the Cloud Function API directly

Direct API calls create tight coupling — if the Cloud Function is slow or down, the publisher blocks or loses data. Pub/Sub decouples them completely. The publisher never waits for processing to complete.

Streaming inserts over batch loading

Alternative Collect events in memory, batch-load to BigQuery every N minutes

Streaming inserts give 1-2 second latency to queryable. Batch loading gives N-minute latency. For a real-time dashboard, streaming inserts are necessary. The tradeoff is higher per-row cost, but at this event volume it's negligible.

FastAPI over direct BigQuery connection in frontend

Alternative Dashboard queries BigQuery directly from the browser

Direct BigQuery access from a browser would expose credentials and bypass access control. FastAPI acts as a secure proxy — it holds the credentials server-side, applies business logic (the 30% alert threshold), and returns only the data the frontend needs.

DATE partitioning over no partitioning

Alternative Store all rows in a single unpartitioned table

Without partitioning, every query scans all rows ever ingested — cost grows linearly with data volume. With DATE partitioning, a query filtered to the last 24 hours scans only today's and yesterday's partitions regardless of total data size.

Section 8 — Outcomes and Results

Quantitative Results

Metric	Result
Total events ingested	2,079+ events across multiple publisher sessions
End-to-end latency	Under 2 seconds (publish → BigQuery queryable)
Components tracked	6 (checkout, search, product-listing, recommendations, ad-banner, user-profile)
API endpoints monitored	5 (/api/products, /api/checkout, /api/search, /api/user, /api/recommendations)
Highest error rate detected	/api/products at 37.5% — auto-alert fired
UI vs API event split	60.8% UI interactions, 39.2% API requests
Dashboard refresh rate	Auto-refresh every 10 seconds
API endpoints exposed	5 REST endpoints + Swagger UI docs
Pipeline reliability	Dead letter queue configured, 0 events lost
Cloud Function status	ACTIVE — Gen2, Python 3.11, us-central1

What Was Learned

- IAM failures are silent by default. When the Eventarc trigger lacked the run.invoker role, Pub/Sub messages were accepted but the Cloud Function was never invoked — no error, no log, just 0 rows in BigQuery. Always verify IAM before assuming a pipeline is broken elsewhere.
- Pub/Sub decoupling is genuinely useful. During Cloud Function redeployments, the publisher kept running. Messages queued in Pub/Sub and were processed in order when the function came back up. This durability is invisible until you need it — then it's everything.
- p99 latency matters more than average. Average latency of 400ms can hide a p99 of 1800ms — meaning 1 in 100 users waits nearly 2 seconds. The analytics query tracking p50/p95/p99 per endpoint is the correct signal for capacity planning.
- Cloud Functions cold start is real. First invocation after an idle period takes approximately 2 seconds. For latency-sensitive pipelines, setting min-instances=1 eliminates cold start at the cost of a few cents per month.
- BigQuery streaming insert cost vs batch. Streaming inserts cost \$0.01 per 200MB of data. For this project that's negligible. At very high volume (billions of events), the cost tradeoff shifts toward micro-batching.

Section 9 — Real-World Connections

This is not an academic project. Every major technology company runs a version of this pipeline. Understanding these connections helps explain why the project was worth building and demonstrates systems thinking beyond the code.

Rakuten Advertising

Tracks ad banner clicks, product views, and API performance across publisher networks. The pipeline feeds dynamic pricing (more clicks = higher CPM) and recommendation engines (users who clicked ad X also bought Y). This project mirrors exactly what the Rakuten internship role involves.

Uber

Surge pricing is a real-time event pipeline. Event volume (ride requests) per geographic zone per minute feeds a pricing model that adjusts the multiplier automatically. p99 latency monitoring on the payment API triggers automatic rollback when it breaches threshold.

DoorDash

Real-time order request volume by zone determines Dasher positioning. The pipeline predicts demand 20 minutes ahead so Dashers are pre-positioned before the lunch rush — not scrambling to catch up after it starts.

Netflix

Every scroll past a thumbnail, every hover, every play event is captured. This data computes A/B test results on thumbnail click rates, drives the recommendation algorithm, and determines which content to commission next.

Swiggy / Zomato

During IPL evening matches, event volume spikes 5-10x in specific cities. The pipeline detects the spike and throttles new orders to restaurants that cannot handle the volume — preventing delivery SLA breaches before they happen.

Datadog / PagerDuty

These billion-dollar companies are built entirely on this idea. Collect events, compute metrics, detect anomalies, alert on-call engineers. The auto-alerting logic in this project (error rate > 30% triggers notification) is the same fundamental mechanism.

Section 10 — How to Explain This in an Interview

For a Non-Technical Recruiter (60 seconds)

Script

I built a system that watches what users do on a product in real time — every click, every purchase attempt, every error — and makes that data instantly visible on a live dashboard. Companies like Uber and Netflix use the same approach to make decisions about pricing, scaling, and what features to build next. Mine processes 2,000+ events with results visible in under 2 seconds, and the dashboard is live right now if you want to see it.

For a Technical Engineer (2-3 minutes)

Script

I built an event-driven pipeline on GCP. Python publisher generates UI and API events, publishes to Pub/Sub every 500ms. Cloud Function Gen2 is triggered via Eventarc on each message — it decodes the base64 payload, validates fields, normalizes the timestamp, and writes to BigQuery using streaming inserts. The table is DATE-partitioned and clustered by component and event_type for cost-optimized queries. Dead letter queue is configured on the Eventarc subscription with max 5 delivery attempts. FastAPI on Cloud Run exposes 5 endpoints including auto-alerting when endpoint error rate exceeds 30%. A Chart.js dashboard polls the API every 10 seconds. End-to-end latency is under 2 seconds. Happy to walk through any specific component.

Common Interview Questions and Answers

Q: Why Pub/Sub instead of calling the Cloud Function directly?

A: Direct calls create tight coupling — if the function is slow or down, the publisher blocks or loses data. Pub/Sub decouples them completely. The publisher never waits for processing. Messages queue automatically during outages and replay when the consumer recovers. This is the same reason Kafka is used at companies processing billions of events.

Q: How does the dead letter queue work?

A: The dead letter policy is configured on the Eventarc subscription — the subscription that connects Pub/Sub to the Cloud Function. If the function fails to acknowledge a message after 5 attempts (throws an exception, times out, crashes), Pub/Sub forwards the message to a separate dead letter topic instead of retrying indefinitely or dropping it. This prevents one bad event from blocking the entire queue.

Q: Why partition the BigQuery table by date?

A: Without partitioning, every query — even 'show me the last hour' — scans all rows ever ingested. BigQuery charges per byte scanned. DATE partitioning means a query for the last 24 hours only scans today's and yesterday's partition, regardless of how many months of data exist. Clustering on top of partitioning adds a secondary sort within each partition, further reducing scan size for queries filtered by component.

Q: How did you handle IAM?

A: GCP's IAM model requires explicit roles for every action across every service. The pipeline-sa service account needs: pubsub.publisher (publish events), pubsub.subscriber (receive via Eventarc), bigquery.dataEditor (write rows), bigquery.dataViewer + jobUser (read for FastAPI), run.invoker (Eventarc → Cloud Run), eventarc.eventReceiver (accept triggers). The biggest lesson: IAM failures are silent. Missing run.invoker caused messages to be received by Pub/Sub but the Cloud Function was never invoked — no error, just 0 rows in BigQuery.

Q: How does the auto-alerting work?

A: Two layers. The FastAPI /events/errors endpoint computes real-time error rates from BigQuery and returns alert: true plus the flagged_endpoints list when any endpoint exceeds 30%. This powers the dashboard alert banner. The second layer is a Cloud Monitoring alert policy that watches Cloud Run request metrics — when non-2xx rate exceeds 30% for 60 seconds sustained, it fires an email notification.

Q: What would you do differently at 10x the scale?

A: Three changes: (1) Switch from streaming inserts to micro-batching (collect 100 events, load as a batch) to reduce BigQuery insertion cost. (2) Add min-instances=1 to the Cloud Function to eliminate cold start latency. (3) Add a Redis cache in front of the FastAPI endpoints so the dashboard polls Redis instead of hitting BigQuery on every 10-second refresh — BigQuery queries have ~1-2 second latency which limits how fast you can refresh.

Live Project Links

- GitHub: github.com/SanketJanger/usage-analytics-pipeline
- Live Dashboard: usage-analytics-api-518291172957.us-central1.run.app/dashboard
- API Docs: usage-analytics-api-518291172957.us-central1.run.app/docs
- Health Check: usage-analytics-api-518291172957.us-central1.run.app/health